UNIVERSITY OF CAMBRIDGE
Department of Computer Science and Technology

May 15, 2026

Efficient Coinductives through State-Machine Corecursors

William Sørensen

Gonville & Caius College

Submitted in partial fulfilment of the requirements for the Computer Science Tripos, Part II

Declaration

I, William Sørensen of Gonville & Caius College, being a candidate for the course, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose.
Signed: William Sørensen
Date: May 15, 2026

Proforma

Word count: 5068

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit.

Contents

1 Introduction 9
1.1 Use cases 1
1.1.1 Infinite datastructures 1
1.1.2 Compiler semantics 1
1.1.3 Safe webservers 1
2 Preparation 2
2.1 Dependent type theory 2
2.2 Lean 2
2.3 Polynomial functors 4
2.3.1 Common polynomials 4
2.4 Recursion and Inductives 5
2.5 Corecursion and coinductives 5
2.5.1 In OCaml 5
2.5.2 Streams 6
2.5.3 The M-type 6
2.5.4 State-machine encoding 7
2.6 Inductive predicate 8
2.7 Coinductive predicate 8
2.8 Bisimilarity 8
2.8.1 Coinduction principle 9
2.9 Methodology 9
2.10 Tools used 9
2.11 Requirement Analysis 9
2.11.1 State machine encoding (Core) 9
2.11.2 Non-termination-monad (Core) 10
2.11.3 ABI Type (Extention) 10
2.11.4 Interaction Trees (Extention) 10
2.11.5 Futumorphic productivity (Extention) 10
3 Implementation 12
3.1 Repository overview 12
3.2 Common 14
3.2.1 Transiteration 14
3.2.2 Expanding the progressive approximation theory 15
3.3 Stream implementation 16
3.4 State machine encoding of M-Types 16
3.4.1 PreM 16
3.4.2 Writing for performance 16
3.4.3 A coinduction principle on polynomials 16
3.5 Proving the equivalence 17
3.6 The ABI Type 17
3.6.1 Weak univalence 19

3.6.2 Relation to Shrink and further universe transforming types ... 19
3.7 NTMonad ... 19
3.8 Interaction trees ... 19
3.8.1 Weak bisimilarity ... 19
3.8.2 A formally verified compiler ... 19
4 Evaluation ... 20
4.1 Performance between SME and PA ... 20
5 Conclusions ... 22
Bibliography ... 23
6 Appendicies ... 25
6.A Graphical p-functors ... 27
6.A.A Common pfunctors ... 28
7 Proposal ... 30

List of figures

Figure 1 Inference rules for Π and Σ types . . . 2
Listing 1 Definition for `Acc` taken from [1]'s GitHub . . . 3
Figure 2 Gcd as Source, Kernel and LCNF . . . 3
Figure 3 Projection polynomial . . . 4
Figure 4 Constant polynomial . . . 4
Figure 5 Product polynomial . . . 5
Figure 6 Sum polynomial . . . 5
Listing 2 Definition of list in OCaml . . . 5
Listing 3 Definition of colist in OCaml . . . 6
Figure 7 Agreement of two trees of height 1 and 2 . . . 7
Listing 4 PA Stream in lean . . . 7
Figure 8 Evenness inductive predicate, from first principles . . . 8
Figure 9 Import graph of the lean component of the project as of 2026-04-04 . . . 13
Figure 10 . . . 14
Listing 5 Transliteration between universes of types . . . 15
Figure 11 Operations on ABI . . . 18
Figure 12 Weak univalence up to shrink . . . 19
Figure 13 Performance of SME, PA, and Big representation . . . 20
Listing 6 . . . 21
Figure 14 The head and child components of $P\alpha_0...\alpha_i$ . . . 27
Figure 15 An object of $P\alpha_0...\alpha_i$ . . . 27
Figure 16 Definition of mapping of mvpfunctors, in this case specalised to $P\alpha_0...\alpha_i$ . . . 27
Figure 17 Definition of $\mathrm{const}_i T$ . . . 28
Figure 18 By initial hom uniqueness,
the object is invariant under mapping. . . . 28
Figure 19 Definition of prj_n . . . 29
Figure 20 By initial hom uniqueness, mapping only changes the nth argument. . . . 29

Chapter 1: Introduction

1.1 Use cases

1.1.1 Infinite datastructures

Often, a problem in programming requires leaving the finite. In cases we want to maintain safety properties of the language. For example:

- We cannot use termination as a critaria for infinite structures, so how do we maintain the totallity of the language.
- One would also want functions written for a language like this to have the expected runtime; we want a constant time to unfold.
- We want an interface that lets us unfold and reasona bout definition just how we would any other function.

A concreate example of a usecase for this would be reading from a UNIX file. Since we can for example have a program `cat` which echos out all it is fed, then what would the output of `yes | cat` be? This would have to be a stream.

1.1.2 Compiler semantics

Recalling L1[1], we gave it a big-step operational semantic. We later tried to implement an interpreter for this semantic to execute L1 programs. This required us to prove quite a few substantial coherences. An alternative way we could have tried would be to write down an interpreter, and simply use this as the semantic. Most proof-assistants would not allow this though, as they require function totallity. So we would want:

- A way to write an interpreter of a partial language, and be able to use this interpreter as a semantic in formal software.
- We would also want the asymptotics of the interpreter to be the same as the source language.
- We want to be able to encode effects in the langauge in addition to just non-termination.

1.1.3 Safe webservers

[1] TODO: Add reference to sem of prog

Chapter 2: Preparation

2.1 Dependent type theory

We can start by adding a type for Types, giving us a higher order logic. Recall back to discrete maths [2], how functions between sets $A, B : \mathbf{Type}$, $A \to B$ correspond with a form of arbitrary product, $B^A \equiv \prod_{x:A} B$. We can generalize this by changing B to be a function $B : A \to \mathbf{Type}$, then letting us write the Π-type[2] $\prod_{x:A} B(x)$. We can also do the dual construction, and get Σ-types. The inference rules for these can be seen in Figure 1. These rules should be familiar as they are the analogous to the rules for $\forall, \exists$ in the 1st order sequent calculus, their relationship can be seen in Equation 1. If you are unfamiliar with dependent types, I recommend viewing them using this relationship. For a more in depth look at dependent type theory, read [3].

$$\frac{\begin{array}{c} A : \mathbf{Type} \\ B : A \to \mathbf{Type} \end{array}}{\prod_{x:A} B(x)}\,\Pi\mathrm{f}$$

(a) Π-forming rule

$$\frac{\Gamma, x : A \vdash v : B(x)}{\Gamma \vdash \lambda x.v : \prod_{x:A} B(x)}\,\Pi\mathrm{i}$$

(b) Π-intro rule

$$\frac{\begin{array}{c} \Gamma \vdash f : \prod_{x:A} B(x) \\ \Gamma \vdash v : A \end{array}}{\Gamma \vdash fv : B[x \ / \ v]}\,\Pi\mathrm{e}$$

(c) Π-elimination rule

$$\frac{\begin{array}{c} A : \mathbf{Type} \\ B : A \to \mathbf{Type} \end{array}}{\sum_{x:A} B(x)}\,\Sigma\mathrm{f}$$

(d) Σ-forming rule

$$\frac{\begin{array}{c} \Gamma \vdash a : A \\ \Gamma \vdash b : B[x \ / \ a] \end{array}}{\Gamma \vdash \langle a, b, \rangle : \sum_{x:A} B(x)}\,\Sigma\mathrm{i}$$

(e) Σ-intro rule

$$\frac{\begin{array}{c} \Gamma \vdash v : \sum_{a:A} B(a) \\ \Gamma \vdash a : A, b : B[x \ / \ b] \vdash x : C \end{array}}{\Gamma \vdash \text{let } \langle a, b \rangle := v \text{ in } x : C}\,\Sigma\mathrm{e}$$

(f) Σ-elimination rule[3]

Figure 1: Inference rules for Π and Σ types

$$\Pi, \Sigma \underset{\text{Categorification}^4}{\overset{\text{propositional truncation}}{\rightleftharpoons}} \forall, \exists \qquad 1$$

2.2 Lean

Lean [1] is a popular dependently typed proof assistant, it is famous for a few reasons.

1. It has Mathlib [5], one of the world's largest repositories of formalized mathematics.
2. It is a proof assistant made for (classical) mathematicians.
3. It has a rich tactic language in which most proofs are written.
4. It has a compiler, and can be used as a general purpose programming language.

We will mainly focus on point 4., as a consequence of this point, we sometimes have a tension between the logical and computation sides of a proof. A classic example of where this occurs is

[2]Some call this a 'dependent product', or 'dependent function', but this sometimes confuses people so I will simply call it either a Pi, or a forall.

[3]Technically this isn't the most general eliminator for Sigma's, as C can depend on the content of the type. For symmetry and simplicity I omit this.

[4]Steve Awodey [4] calls this 'categorification', as it is only one direction of Curry-Howard. I know Equation 1 is just a more verbose way of writing 'Curry-Howard', but I find it useful to be verbose here.

in the elimination principle for strong-induction[5].

Lean requires all induction to be structural, so to do induction over a general well-founded relation, we need some inductive (Section 2.4), Lean calls this `Acc` and defines it as seen in Listing 1. On this they define a function `WellFounded.fixF` which allows for strong induction. This function is at least quadratic in its runtime, therefore as a consequence the kernel takes this quadratic runtime to execute.

```
inductive Acc {α : Sort u} (r : α → α → Prop) : α → Prop where
  /--
  A value is accessible if for all `y` such that `r y x`, `y` is also accessible.
  Note that if there exists no `y` such that `r y x`, then `x` is accessible.
  Such an `x` is called a _base case_.
  -/
  | intro (x : α) (h : (y : α) → r y x → Acc r y) : Acc r x
```

Listing 1: Definition for `Acc` taken from [1]'s GitHub

It would be unacceptable for a program using strong recursion to be quadratic for a general purpose programming language, therefore the compiler decides to simply not compile the definition using `fixF`, and generates the expected intermediate representation.

We can see that given the source-code Figure 2a, it generates a definition Figure 2b with `Nat.fix` (a variant of `fixF`), but in the output lean intermediate-representation we get Figure 2c, which does not contain any mention of `Nat.fix`, but rather the expected recursive call.

```
set_option trace.Compiler.result true in
@[semireducible]
def gcd (a b : Nat) : Nat :=
  if h' : b = 0 then a
  else
    have : a % b < b := Nat.mod_lt a
      (Nat.pos_of_ne_zero h')
    gcd b (a % b)
termination_by b
```

(a) GCD source code

```
/--
info: def gcd._unary : (_ : Nat) ×' Nat →
Nat :=
WellFounded.Nat.fix (fun x ⇒ PSigma.casesOn
x fun a b ⇒ b) fun _x a ⇒
  PSigma.casesOn (motive := fun _x ⇒
    ((y : (_ : Nat) ×' Nat) → InvImage (fun
x1 x2 ⇒ x1 < x2) (fun x ⇒ PSigma.casesOn x
fun a b ⇒ b) y _x → Nat) →
      Nat)
    _x
    (fun a b a_1 ⇒
      if h' : b = 0 then a
      else
        have this := …;
        a_1 (b, a % b) …)
    a
-/
#guard_msgs in
#print gcd._unary
```

(b) generated kernel code

```
[Compiler.result] size: 9
  def gcd a b : tobj :=
    let _x.1 := 0;
    let _x.2 := Nat.decEq b _x.1;
    cases _x.2 : tobj
    | Bool.false ⇒
      let _x.3 := Nat.mod a b;
      dec a;
      let _x.4 := gcd b _x.3;
      return _x.4
    | Bool.true ⇒
      dec b;
      return a
```

(c) Generated intermediate representation

Figure 2: Gcd as Source, Kernel and LCNF

This general trick was what inspired me to start this project, as it demonstrates that there is precident in Lean4 to have seperate logical and executional models. This is further highlighted by the existence of the `@[implemented_by ... ]` attribute, which lets you off-load the runtime behaviour of a function to another possibly unsafe definition. The semantics of `@[implemented_by]` is simply calling the alternative function when not evaluation in the kernel. Safety of programs using `@[implemented_by]` is a very complex endeavour to find, as lean has a tactic called `native_decide` that runs the generated code outside of the kernel. I have been told by some of the maintainers of [5], that all definitions that use `@[implemented_by]` should also be `@[irreducible]`.

[5]This is now, for the case of natural numbers, fixed as of lean 4.27. https://www.youtube.com/watch?v=LOUbbiV0mWc https://lean-lang.org/doc/reference/latest/releases/v4.27.0/

2.3 Polynomial functors

I will give two definitions now in the next paragraphs, these will be unary and (multivariate) polynomial functors respectively.

Morally polynomial functors can be thought of as types generic in some set of arguments, with a collection of constructors (the head type), where the children correspond to how many of the polymorphic argument are wanted. The formal definition is as follows:

A (multivariate) polynomial functor on Type[6] is: a head-type, along with a (collection of) child family(ies) parameterised by the head. An object of a polynomial functor is a select head type, and the functions form the corresponding child(ren) parameterised by the head as seen in Equation 2[7].

$$(h : H) \times c_h \to \alpha \equiv \sum_{h:H} \alpha^{c_h} \tag{2}$$

Polynomial functors have all finite products (Figure 4 Figure 5) and coproducts (Figure 4 Figure 6). Polynomial functors are also closed under (co)fixed points so I will write them using a notation a la inductives, in the next examples I have an ellipsis, in reality multivariate polynomials operate on type-vectors which come from the category formed by the products Type^n for some n, these are an absolute pain to work with as I will discuss in the implementation[8]. The justification for writing pfunctors using an inductive notation can be found in A. Keizer [7] and J. Avigad, M. Carneiro, and S. Hudon [8].

2.3.1 Common polynomials

Some examples of common polynomials can be seen in the following figures, one crucial polynomial I am not mentioning which is maybe the most useful one but also most complex one, is composition. The composition polynomial works exactly like composition in primitive recursive functions, it takes a n-polynomial, and n lots of m-polynomials, and outputs an m-polynomial. The polynomial notation for this can be found in [5].

We can now using these functors make a polynomial compiler from any syntax of the form `P :: var | P × P | P + P | const n` where `var :: x | y | ...`, this compiler is trivially structurally recursive on the syntax, simply when you see a var do `Prj` (Figure 3), a const `Const` (Figure 4), and for the binary operators first a composition then the expected polynomial.

```
inductive Prj (i : Fin n) (a_1 ... a_n : Type)
  where
  | mk (t : a[i])
```

(a) Inductive Notation

$$\lambda(i : \underline{n}).\langle 1, \lambda j \quad h.\ \text{if } i = j \text{ then } 1 \text{ else } 0\rangle \tag{3}$$

(b) Polynomial

Figure 3: Projection polynomial

```
inductive Const (T : Type) (a_1 ... a_n : Type)
  where
  | mk (t : T)
```

(a) Inductive Notation

$$\lambda(T : \text{Type}).\langle T, \lambda i \quad h.0\rangle \tag{4}$$

(b) Polynomial

Figure 4: Constant polynomial

[6]For the readers familiar with category theory, simply think of categorical dependent products, and (co)limits when I use the set-theoretic language.

[7]More can be read on the ncatlab article [6], we restrict ourselves to Set, so I recommend skipping to that section.

[8]TODO ADD EXACT SECTION

```
inductive Prod (A B : Type) where
  | mk : A → B → Sum A B
```

(a) Inductive Notation

$$\langle 1, \lambda i \quad ().1 \rangle \tag{5}$$

(b) Polynomial

Figure 5: Product polynomial

```
inductive Sum (A B : Type) where
  | inl : A → Sum A B
  | inr : B → Sum A B
```

(a) Inductive Notation

$$\langle \mathbb{B}, \lambda \begin{pmatrix} 0 & \text{true} & \mapsto & 1 \\ 1 & \text{false} & \mapsto & 1 \\ _ & _ & \mapsto & 0 \end{pmatrix} \rangle \tag{6}$$

(b) Polynomial

Figure 6: Sum polynomial

2.4 Recursion and Inductives

We are all familiar with algebraic datatypes from OCaml. These are structures freely generated from a set of constructors. The classic example is a list, given as two constructors, `val cons : 'a → 'a list → 'a list` and `val nil : 'a list`, and a way to consume lists `val fold : 'a → ('b → 'a → 'a) → 'b list → 'a` (in lean we say `rec` instead of `fold`). We write this as seen in Listing 2. This is what is known as an inductive datatype. Inductive data-types are well-founded trees[9] any function that terminates, must by definition return at most a (unbounded) finite tree of constructors. This means fold is guaranteed to terminate too. All (structural) recursive functions can be compiled into a call to fold[10]. We also have two operations mk and dest. These form an equivalence (J. Lambek [9] Corollary 2.4), giving us the result that the fix-point is exactly that $F(\mu F) \simeq \mu F$.

```
type ('a, 'b) listF = Nil | Cons of 'a * 'b
type 'a ilist = Mk of (('a, 'a ilist) listF)

let mk x = Mk (x)
let dest = function | Mk v → v
let rec fold bh ih = function
    | Mk(Nil) → bh | Mk(Cons(h, t)) → ih h (fold bh ih t)
```

Listing 2: Definition of list in OCaml

2.5 Corecursion and coinductives

Coinductive datatypes are datatypes we allow to be infinitely deep. Where inductive datatypes have a `fold : (('a, 'b) f → 'b) → (('a, 'g) f as 'g) → 'b` (rec) to consume them, coinductives have an `unfold : ('b → ('a, 'b) f) → 'b → (('a, 'g) f as 'g)` (corec) to produce them. The intuition for this if you have never seen `unfold`'s or coinductives, simply view it as if a fix F yields an X, then cofix F yields a lazy X[11]. Another way to view coinductives are as a state-machine, here the corecursor takes center stage, we can say a coinductive datatype is a datatype generated by some corecursor state-machine.

2.5.1 In OCaml

We can think back to what we did in Foundations of Computer Science, we made lazy lists simply by thunking recursive occurrences, this thunking allows infinite structures to exist in finite memory. This happens because OCaml is CBV. If we were in Haskell (or any call by

[9]In OCaml, data-types do not correspond directly to inductives since we can have non-well-founded trees, an example is `let rec degen = Cons((), degen)`,

[10]NOTE FOR REVIEW: does this need a citation?

[11]I. e. fix $(1 + (A \times -))$ gives lists, then cofix $(1 + (A \times -))$ gives lazy lists.

name / push-value language), the definition of icolist would be exactly that of ilist. Then all we need to do is add a definition of `unfold`, `mk` and `dest`. All these can be seen in Listing 3. We have a coinductive version of Lambek's theorem now too.

```
type 'a icolist = CMk of (unit → ('a, 'a icolist) listF)

let mk x = CMk (fun _ → x)
let dest = function | CMk v → v ()
let rec unfold gen v = CMk (fun _ →
    match gen v  with
    | Nil → Nil
    | Cons(h, t) → Cons(h, unfold gen t))
```

Listing 3: Definition of colist in OCaml

The keen eyes will notice and call out that `unfold` is not structurally recursive, nor even decreasing at all. This is the key difference between inductives and coinductives; coinductives can be 'infinitely big'. Though it is not decreasing, assuming gen is terminating it is *productive* (think of this as coterminating). As termination is having a complete structure in finite time, productivity is growing in finite time.

2.5.2 Streams

Streams are similar to lists, the only difference is they have no 'nil' constructor. As one might expect there cant exist any inductive of this form[12]. Though this is a perfectly well-defined coinductive. Streams are also common in general purpose programming, Java has Stream, Rust and Python have Iterator. This backs up our use cases put forward in the introduction. We will give concrete implementations of them for each of the encodings as follows.

2.5.3 The M-type

The M type is the name given to the cofixed-point I mentioned[13] possibly infinitely deep trees in which each layer is an unfolding of F (dual of Lambek's [9]). This means the implementations must have both: A corecursor $\text{corec} : \{\alpha : \text{Type}\} \to (f : \alpha \to F\alpha) \to \alpha \to MF$, and a destructor $\text{dest} : \text{M } F \to F(MF)$. These are the what we mentioned above, often we also want to ship a way to `mk` a new layer, but we can define this from the two operators above.

When it comes to encoding M-types in lean, there are a few different approaches. We will focus on two encodings of M-type's, the State-Machine encoding Section 2.5.4 and Progressive Approximation Section 2.5.3.1 (as seen in [5]).

2.5.3.1 Progressive approximation encoding

A simple way to encode M-types, are as functions emitting trees that must at every level 'agree', this corresponds to the expected limit[14] over the natural number poset category from the dual of J. Adámek [10]'s "Free" algebra algorithm. Agreement is given by them being the same up to the previous depth as seen in Figure 7. I like to think about agreement as the structure 'crystalizing', since it can only 'grow' from the ends. This is really easy to implement in lean.

[12]See the following lean

```
inductive SS (α : Type) where | cons : α → SS α → SS α
def SS.toEmpty : SS α → Empty | .cons _ tl ⇒ toEmpty tl
```

[13]Often phrases as types who's semantics are terminal F-coalgebra.

[14]TODO: is this a colimit

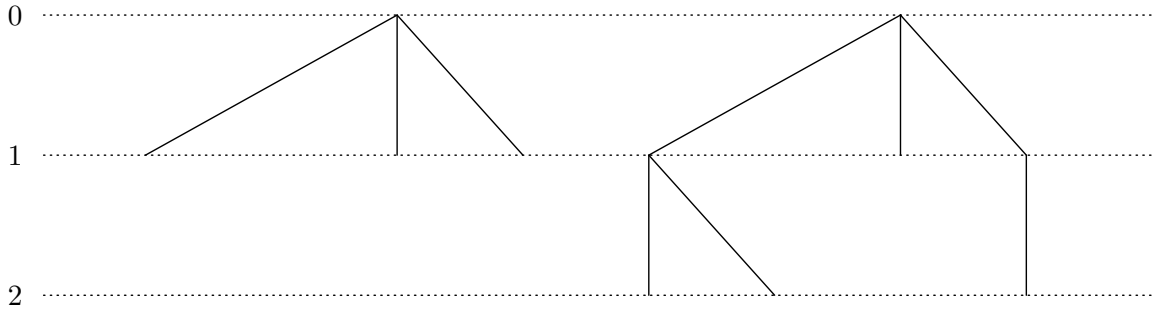


Figure 7: Agreement of two trees of height 1 and 2

2.5.3.1.1 In lean

In Lean, this encoding is very easy to set up. It can be seen in Listing 4. The main component which is hard to define is a coinduction theorem (Section 2.8.1), M. P. Fiore [11] is a very insightful read on this topic. (I recommend reading up to Section 6). Luckily for us we dont need to worry about implementing and proving this as [5] already provides an implementation of this. This was ported to Lean4 as part of [7].

A major problem with this implementation is the complexity of access. As you can see in Listing 4 in the definition of `corec.o`, we have to regenerate the entire tree to get a corecursor to the next depth. There are a few different approches to solve this that we will compare and contrast in Section 4.1

```
def StreamF (α ρ : Type _) := α × ρ

inductive StreamF.Crystal {α : Type} {base : Type} : {n : Nat} → {m : Nat}
    → n.repeat (StreamF α) base → m.repeat (StreamF α) base → Prop where
  | base  : @Crystal _ _ 0 (m + 1) b v
  | consS : @Crystal _ _ n m t₁ t₂ →
      @Crystal _ _ (n + 1) (m + 1) ⟨h₁, t₁⟩ ⟨h₁, t₂⟩

structure PA (α : Type _) : Type 0 where
  obj : (n : Nat) → n.repeat (StreamF α) Unit
  cryst : ∀ n, StreamF.Crystal (obj n) (obj (n + 1))

namespace PA
def corec.o (gen : ρ → StreamF α ρ) (v : ρ) : (n : Nat) → n.repeat (StreamF α) Unit
  | 0 ⇒ .unit | n + 1 ⇒ (gen v).map id (o gen · n)
def corec (gen : ρ → StreamF α ρ) (init : ρ) : PA α where
  obj := corec.o gen init
  cryst n := by
    induction n generalizing init
    · exact .base
    case succ n ih ⇒
      unfold corec.o
      rcases gen init with ⟨a, b⟩
      change StreamF.Crystal (n := n+1) (m := n+2) ⟨a, corec.o gen b n⟩ ⟨a, corec.o gen b (n + 1)⟩
      exact StreamF.Crystal.consS (h₁ := a) (ih b)

def dest (x : PA α) : StreamF α (PA α) := ⟨
  (x.obj 1).1,
  (x.obj ·.succ ▷.snd),
  fun n ⇒ by
    have := x.cryst (n+1)
    dsimp
    generalize (x.obj (n + 1)) = a, (x.obj (n + 1 + 1)) = b at this;
    rcases a; rcases b; rcases this with (_|ih)
    exact ih
⟩
end PA
```

Listing 4: PA Stream in lean

2.5.4 State-machine encoding

The state-machine encoding is the naïve way to implement the terminal coalgebra. Given some polynomial functor F, the state-machine encoding is given by: some type $\alpha : \text{Type}$, a function $f : \alpha \to F\alpha$, and some witness $a : \alpha$. Once this is done there are 3 key problems: 1) The object

constructed is only weakly terminal, 2) the object created has no coinduction principle, 3) the object resides in a higher universe. Let us first focus on the first two problems (the 3rd problem is tackled in Section 3.6), both of these are solved by quotienting over bisimilarity. The reason this works has to do with how bisimilarity 'hides' the generating type, making the only operation you really can do to access the data a destructure.

2.6 Inductive predicate

An inductive predicate is a least fixed point; it is a universal quantification over all relations of a given shape. An example of an inductive predicate is whether a number is even. We begin by defining this as a higher order predicate, the shape, we define this as seen in Figure 8. It can be shown that this definition is equal to the one generated by the expected lean code.

```
def IsEven.Shape (P : Nat → Prop) : Prop :=
  P 0 ∧ ∀ n, P n → P (n + 2)
def IsEven (n : Nat) : Prop :=
  ∀ P, IsEven.Shape P → P n
def IsEven.z : IsEven 0 := fun P hP ⇒ hP.left
def IsEven.ss n (h : IsEven n) : IsEven (n+2) :=
  fun P hP ⇒ hP.right n (h P hP)
```

(a)

$$\begin{aligned} \text{shape}_P &\triangleq P0 \wedge \forall n, Pn \to P(n+2) \\ \text{Even } (n) &\triangleq \forall P, \text{shape}_P \to Pn \\ e_0 : \text{Even } 0 &\triangleq \lambda P \text{ hP}.\pi_1(\text{hP}) \end{aligned} \tag{7}$$

(b)

Figure 8: Evenness inductive predicate, from first principles

2.7 Coinductive predicate

Given what we saw in the last section and how we have seen the duality for coinductive and inductive data; we now know all we must do is dualize this construction. So we change the universal to an existential, disjuncts to conjuncts, and flip the arrows (each of these swap left and right adjoints). This gives us a proof principal with three components. We now need to define an invariant, we need to proof the invariant satisfy the restriction, and that our value is contained in the invariant.

Proving coinductive predicates is very challenging as it requires coming up with the invariant, this is a kind of proof I initially had very little experience in and took me a while to pick up. An additional difficulty is, if you get the invariant wrong, you just have to start completely again and try to come up with something better. This means that for some of my proofs I hundreds of lines of code that have not made it into the dissertation.

Some languages have facilities to make this better, these are collectively refered to as coinduction-up-to. These are techniques where you can write parts of a proof somewhere in a certian style, then reuse them other proofs. For Rocq a popular library for this is Paco TODO CITE, for which a lot of the harder proofs often use. Lean does not have this feature, and it is out of scope for me to add it.

2.8 Bisimilarity

In this section we will talk about the equivalence relation on co inductive types called bisimilarities. Morally two things are bisimilar if for every possible choice in a structure A there exists a corresponding choice in structure B such that you can repeat this procedure from the new point.

On streams this corrensponds to saying that the heads of the streams are equal and then the tails are bisimilar. As you may note this is not a well founded relation. This means we need to

define it as a coinductive predicate rather than an inductive one. Then the proof principal for this will be giving a relation, showing it is a bisimilary and that it holds on our values.

2.8.1 Coinduction principle

A coinductive type has a conduction principle is bisimilarites implies equality. For the SME this will be harder to show.

2.9 Methodology

Writing in a proof-assistant is quite different than writing in a general purpose language, this is especially the case in Lean as it is a hybrid of both styles. A notable and immediate difference is the prevalence of software testing of programs. Rather than having tests that can be flakey or incomplete, in lean you have checked proofs. From this in a conventional programming language, you can for example follow TDD where you constantly refine the specification with additional tests, which you check as you go through the code. Proof Driven Development (PDD, [12]) perfect for this in Lean after seeing success in Idris. In PDD you work similarly to TDD, but now when you have your function, rather than knowing it works for finite cases, you can be guaranteed that it works. Furthermore, the proofs can serve to enrich the main artifact. This will be seen in the ITree implementation as well as the futumorphic productivity. Lean makes working in this style a delightful experience using constructs such as `#check`, `#print` and `#eval`, one can see exactly what a function is / is doing. These can also be turned more conventional tests using `#guard_msgs in`, which raises a compile error upon an output changing. Lean also has great automation features which I use at many points in the dissertation, these are the unification solver `exact?` and SMT solver `grind`. These both stand as principled (as opposed to LLM Slop) ways to develop programs and proofs that would otherwise be tedious for a human.

2.10 Tools used

To aid the development and communication process I used `git`, hosting the repository on GitHub. I also shipped a compiled version of this dissertation publicly, so my supervisors could read it and have it update live on the web. The dissertation is written in **typst** as it is a feature rich, well documented, and popular TeX alternative.

2.11 Requirement Analysis

To expand on the success criteria given in the proposal, I specified the project using MoSCoW. Here the success criteria have become MUSTs, and improvements have become SHOULD. I have done this for the core and each of the extensions of the project.

2.11.1 State machine encoding (Core)

SM1 The SME Stream MUST be implemented in its high universe form.
SM2 The equivelence between SME Stream and PA Stream MUST be implemented.
SM3 The SME M MUST be implemented in its high universe form.
SM4 The SME M MUST be constant time under destructuring.
SM5 The SME M MUST be able to express the NT monad, Section 2.11.2.
SM6 The equivelence between SME M and PA MUST be implemented.

SS1 The SME M SHOULD have a coinduction principle.

SC1 The SME M COULD have a low universe shifted version Section 2.11.3.
SC2 The SME M COULD have an Interaction tree library Section 2.11.4.
SC3 The SME M COULD have a Productivity Transform Section 2.11.5.
SC4 The SME M COULD be zero cost.

SW1 The SME M WONT have a syntax macro a la [7].
SW2 There WONT be any work towards coinduction-up-to systems.

2.11.2 Non-termination-monad (Core)

NM1 The NTMonad MUST be implemented using the SME.
NM2 The NTMonad MUST have a monadic bind and return.
NM3 The NTMonad MUST be linear in compression.

NS1 The NTMonad SHOULD be a lawful monad.

NC1 The NTMonad COULD be set up to use Section 2.11.5.

NW1 The NTMonad WONT have a syntax macro.

2.11.3 ABI Type (Extention)

AM1 The ABI Type MUST be implemented.
AM2 The ABI Type MUST have an eliminator.
AM3 The ABI Type MUST have the computational behaviour of the higher type.

AC1 The ABI Type COULD be zero-cost.

2.11.4 Interaction Trees (Extention)

IM1 ITrees MUST be implemented.
IM2 ITrees MUST have strong bisimilarity.
IM3 ITrees MUST have bind and return constructs.
IM4 ITrees MUST have KTrees (continuation trees).

IS1 ITrees SHOULD have flow combinators implemented.
IS2 ITrees SHOULD have a coinduction principle.
IS3 ITrees SHOULD be a lawful monad.

IC1 ITrees COULD have weak bisimilarity implemented.
IC2 ITrees COULD have the monadic interpretation.

IW1 ITrees WONT implement the family of effects put forward in [13].

2.11.5 Futumorphic productivity (Extention)

FM1 Futumorphic productivity MUST have a in universe corecursor.
FM2 Futumorphic productivity MUST have an injector.

FS1 Futumorphic productivity SHOULD have proof-principles for unfolding the corecursor.
FS2 Futumorphic productivity SHOULD have proof-principles for the injector.

FC1 Futumorphic productivity COULD have a cross universe corecursor.

FC2 Futumorphic productivity COULD have a universe transliterator.

FC3 Futumorphic productivity COULD have the ability to reason about *closed* trees.

FW1 Futumorphic productivity WONT be as expressive as [14].

Chapter 3: Implementation

This chapter describes the implementation of each of the requirements stated in Section 2.11. I will break it down into a common section, then go by induvidual components. I will also mention which parts are in process of being upstreamed and which parts are already in [5].

unnacc

- rq:sme:fast
- rq:sme:ct

3.1 Repository overview

The Lean component of this repository, and how they relate to each of the requirements can be seen in the list below. Additionally the import graph can be seen in Figure 9

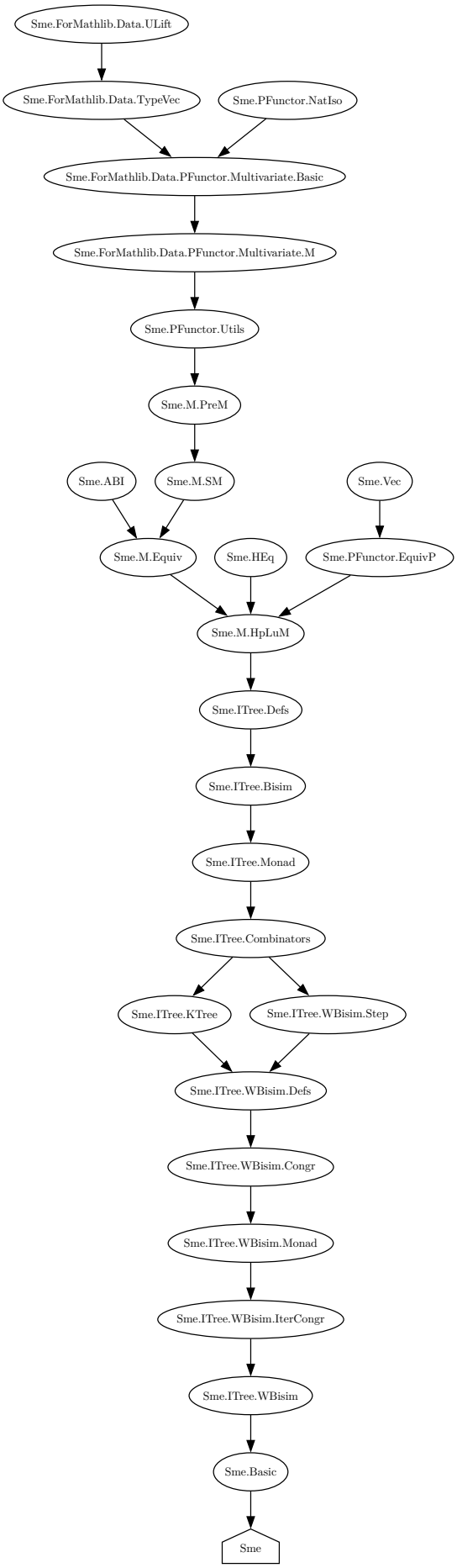
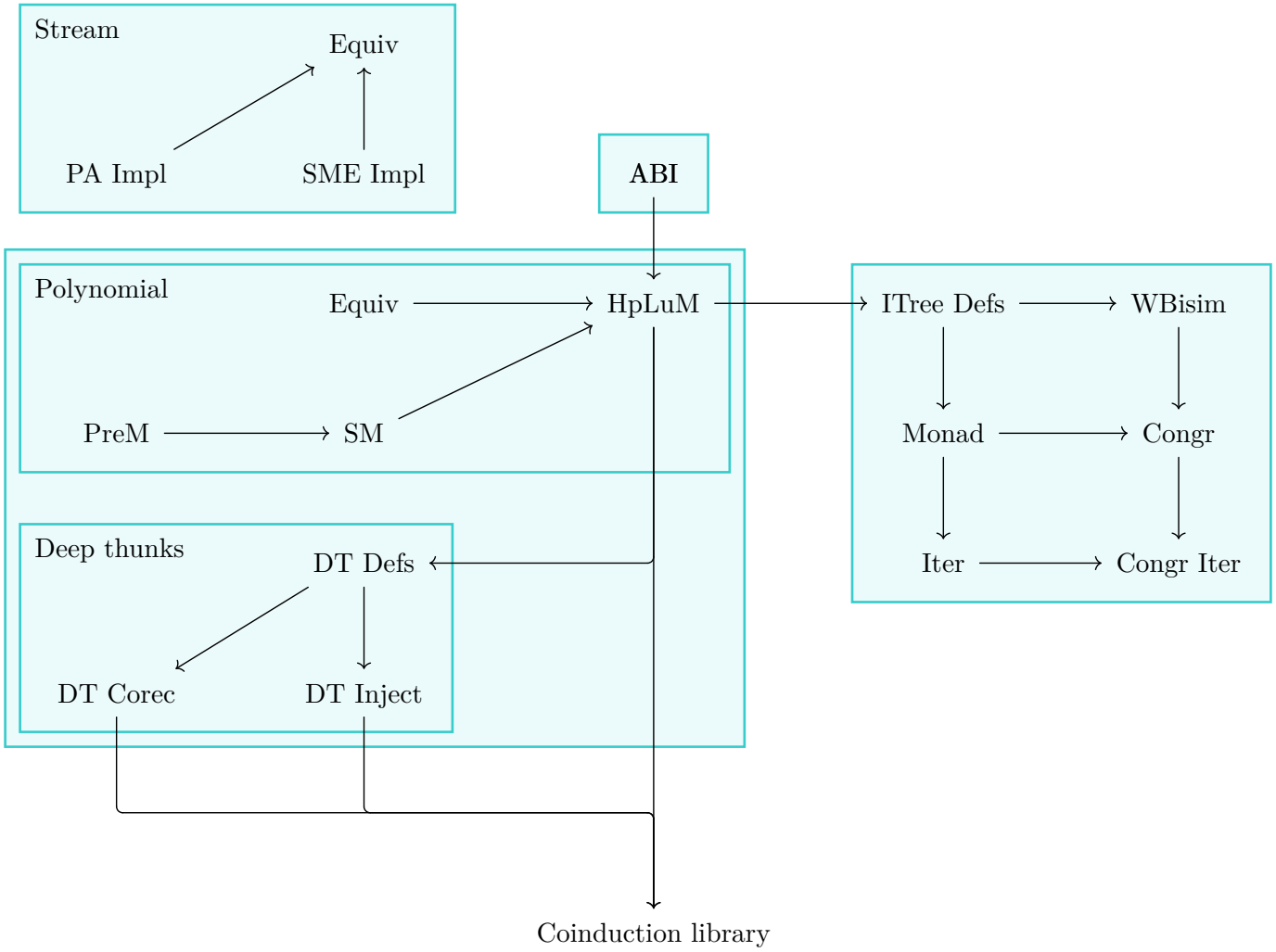


Figure 9: Import graph of the lean component of the project as of 2026-04-04



3.2 Common

3.2.1 Transiteration

A family of functions, that keep solving problems throughout this dissertation are what I call transliterations. Given some parameter span X[15], either in some **Type**, or more commonly over some universe $\mathcal{U}$. We define a transliteration on X as a family of functions $t_{a,b} : Xa \to Xb$, such that they are closed under composition $t_{b,c} \circ t_{a,b} = t_{a,c}$[16], and the self-path is identificaion $t_{a,a} = \mathrm{id}$.

The trivial instantiation of a transliteration is a `cast`[17], here we pick X as equal types, obviously `cast aa = id` and `cast bc ∘ cast ab = cast (ab.trans bc)`. One could even say that a transliteration is a function that behaves like `cast`.

An example we will keep defining is universe transliteration, here we take $X = \mathrm{ULift}_{(-)}$, using this we define the transiteration as seen in Listing 5. This is the closest we get to having omega-types and cumilativity in lean; as long as the function is applied at a usage of ULift it all just

[15]This not necessarily a type, as lean does not have omega-types (Setω from Agda).
[16]This is very similar to saying it's an idempotent, but technically $t_{b,c}$ and $t_{a,b}$ arent the same function.
[17]Or if you read [3], this is called `transport`.

works. We will use this function to define transliteration between universe-lifted polynomials, we will see more of this in Section 3.2.2.1.

```
universe u v w
variable {A : Type u} {x : ULift A} {a : A}

@[inline] def transliterate : ULift.{v} A → ULift.{w} A := ULift.up ∘ ULift.down

@[simp] theorem transliterate_idempotent
    : transliterate ∘ transliterate = transliterate (A := A) := rfl
@[simp] theorem transliterate_noop
    : transliterate x = x                                    := rfl
```

Listing 5: Transliteration between universes of types

Another major usage of a transiteration was used when defining the eliminator for `ABI`, I will discuss this more when we get to Section 3.6. This was where I first discovered transliteration, and made it possible to define a universe-polymorphic eliminator.

3.2.2 Expanding the progressive approximation theory

During the feasability assesment I noticed that, in the current formalised theory of polynomials, the statement wouldn't even type-check. This stemmed from a problem with the corecursive principle for the M type in the old implementation. corec : $\{\alpha : \text{TypeVec}.\{\mathcal{U}\}n\} \to \{\beta : \text{Type } \mathcal{U}\} \to (g : \beta \to P(\alpha ::: \beta)) \to \beta \to M\alpha$[18] [5]. The problem here is that both α and β have to both reside in $\mathcal{U}$. Solving this is done through the next two sections.

3.2.2.1 Universe lifting of polynomial functors.

The main problem caused here comes from the fact that lean isnt cummulative. This means it is impossible to express a universe hetrogouns typevector. In other words $\alpha ::: \beta$ is only typable if $\alpha : \text{TypeVec}.\{\mathcal{U}\}n$ and $\beta : \text{Type } \mathcal{U}$. The natural way of solving this is using the supremum in universe levels you get from ULift : Type $\mathcal{U} \to$ Type $(\max \mathcal{U}\mathcal{V})$. This means we can have $\beta :$ Type $\mathcal{U}$ and $\alpha :$ Type $\mathcal{V}$, then ulift both of them to a common universe ULift $\alpha ::: $ ULift $\beta :$ TypeVec.$\{\max \mathcal{U}\mathcal{V}\}(n+1)$[19].

Noticable the next hurdle we encounter is that PFunctors are restricted to a universe level. Recall the definition from Section 2.3. Observe how for a MvPfunctor.$\{\mathcal{U}\}n$, we require that both the head and child reside in $\mathcal{U}$. This will also cause problems, as looking back at the definition of the corecursor, we will require P to be able to accept ULift $\alpha ::: $ ULift β. If we do not add the ability to lift P, the unifier will force $\mathcal{U} = \mathcal{V}$, thereby invalidating all the work we did in the previous section. Luckily lifting a PFunctor is relatively easy. We define it as ULift $P \triangleq \langle \text{ULift } P.1, \lambda x \mapsto \text{ULift } (P.2x) \rangle$. This works and now we can move on to our goal[20].

3.2.2.2 Generalizing the corecursor

Now with all the work in the previous section, by generalizing corec'[21], we can define corecU : $\{\alpha : \text{TypeVec}.\{\mathcal{U}\}n\} \to \{\beta : \text{Type } \mathcal{V}\} \to (g : \beta \to \text{ULift } P(\text{ULift } \alpha ::: \text{ULift } \beta)) \to \beta \to M.\{\mathcal{U}\}\alpha$. Notably we are able to fit the object into $\mathcal{U}$ (as opposed to in the SME).

The expected diagram using corecU and dest commutes.

[18] https://github.com/leanprover-community/mathlib4/blob/7a60b315c7441b56020c4948c4be7b54c222247b/Mathlib/Data/PFunctor/Multivariate/M.lean#L152-L154

[19] Note we overload ULift as a notation to refer to lifting TypeVecs as well

[20] TODO: Speak with JV / W to see if this might be done in the lit, Ex : Locally presentable and accessable categories Adameck roshiski

[21] Done in PR NUMBER

3.3 Stream implementation

Addressing: SM1, SM2

As proving these equivleneces is extremely challenging I decidede I would start by implementing it for the special case of streams. Streams are the text-book coinductive datatype that most people know as mentioned in Section 2.5. Therefore I expected this to be an easy task to start with to test showing the equivilence.

First I set up a class of preobject, these correspond to the corecurisve principle holding tehir type. Then I defined a bisimilarity relation, I proved this was an equivalence relation. Then I quotiented the preobject with this relation. Once this was done I specialised the PA encoding to streams as seen in FIGURE. I also specialised the base bisimilarity relation for polynomials to be analougous to the one defined for the SME.

Once this was done I started proving the equivalence. The functions of this equivalence correspond to the corecursors parameterised by the destructors of the opposite type as seen in FIGURE. To prove that these functions are inverses, I had to find a relation for the bisimilarity. This was quite challenging but in the end I found an equiality solved it.

Having done this I now knew how to approach the case of the polynomial.

3.4 State machine encoding of M-Types

Addressing: SM1, SM2

Noting the definition of corecU, one might wonder if you could define M from first principles for this. The problem one encounters is one of universes. As seen in the definition above, if one were to define a type whos construcor is directly the corecU definiton, it would hold a $\beta : \text{Type } \mathcal{V}$. This then forces the object to reside in $\text{Type} \max \mathcal{U}(\mathcal{V}+1)$. This is a problem as one loses most closure results as you will be lifting more and more. The main beinifit from this is the performance aspect. Going from reading to a depth n, to a depth $n+1$ is not $\mathcal{O}(1)$ instead of $\mathcal{O}(n)$. This will be seen in Section 4.1. We will henceforth refer to the datatype SME.PreM.

3.4.1 PreM

As we speak about in the M Type is the terminal object in the category of coalgebras. We can see through reasoning (cumilatively) in this category that PreM is weakly terminal. Looking at this category we want to force the incoming morphisms together. This corresponds exactly to quotienting just as we saw for streams.

3.4.2 Writing for performance

Addressing: SM4, SC4

3.4.3 A coinduction principle on polynomials

Addressing: SS1

For an arbitrarry polynomial we can define bisimilarities for its cofix point. Mathlib has a definition for this for the PA encoding [5]. This has the structure as seen in FIGURE. I found this very challenging to work with; Not only does this require using heterogeneous equalities,

it also requires synthesizing three elements. These are the head and two children under the provided head. After struggling with this, I realize it can be solved using the universal property of the terminal object. The exact structure of this can be seen in FIGURE. An example of where this shines is as an alternative to conduction-up-to. Instead of having the ability to extend the relation, we get parts of the way there using automatic solving tactics like simp as seen in FIGURE.

3.5 Proving the equivalence

Addressing: SM6

Proving the equivalence on polynomials is much more challenging than proving it on streams. We know at least from streams, each of the directions will be given by each of their corecursors. One might expect to follow the type signatures and mostly you do, but Lean couldn't help with this, as it had to do higher order unification. When Lean finally type checked one could see the functions are obvious as seen in FIGURE. We now need to prove the expected diagrams commute. When I was proving these equalities I was not very familiar with bisimilarities, and tried tens of relations. Finally I landed on simply forcing their equalities. Once this was found I had to find the three required structures for bisimilarity. I picked the head of the first type and was lucky that they both definitioaly was of the same type. Then to prove the final equalities I proceeded by sigma-, unction extentiality, finally proving the equalities. The other direction followed analougesly.

This was the main deliverable of the project, and would help make high performance low universe M types (HpLuM). Sadly I could not use it directly but instead needed to also make a transliteration REFERENCE. This transliteration helps justify the uniqueness of a low universe M type. I hacked and used the Lean ABI stating this is a noop as mentioned in section REFERENCE TRANS.

I used this to instantiate the ABI type. I then gave it a destructor, a corecursor, and a coinduction principal, I proved the expected unfold lemass and gave it a functorial map. For the rest of this dissipation I will be proving the results about this type.

3.6 The ABI Type

The problem the ABI type tries to tackle is one of abstracting the runtime datatype through functions. Given an isomorphism eq : $\alpha \cong \beta$ for some types α and β, my first try at solving this involved constructing an object ABI $\alpha\beta$, making the following commute:

$$
\begin{tikzcd}
 & \beta \arrow[dd, "\mathrm{mkB}"] \arrow[ddrr, "\mathbb{1}_\beta"] & & \\
\alpha \arrow[ur, equal, "\mathrm{eq}"] \arrow[r, "\mathrm{mkA}"] \arrow[ddrr, "\mathbb{1}_\alpha"] & \mathrm{ABI}\ \alpha\beta \arrow[r, "\mathrm{destB}"] \arrow[dd, "\mathrm{destA}"] \arrow[dr, "\mathbb{1}_{\mathrm{ABI}\ \alpha\beta}"] & & \beta \arrow[dl, "\mathrm{mkB}"] \arrow[ddll, equal, bend left, "\mathrm{eq}"] \\
 & & \mathrm{ABI}\ \alpha\beta & \\
 & \alpha \arrow[ur, "\mathrm{mkA}"] & &
\end{tikzcd}
$$

Figure 11: Operations on ABI

Additionally I had an elimination principle satisfying the two equations below.

```
def elim : {motive : ABI _ _ eq → Type w}
       → (hLog : (z : A) → motive (mkA z))
       → (hCheap : (z : B) → motive (mkB z))
       → (eqA : ∀ z, hLog z ≍ hCheap (eq z))
       → (eqB : ∀ z, hCheap z ≍ hLog (eq.symm z))
       → (v : ABI _ _ eq) → motive v := _

theorem elimLog : {motive : carry → Type w}
       → {hLog : (z : A) → motive (mkA z)}
       → {hCheap : (z : B) → motive (mkB z)}
       → {eqA : ∀ z, hLog z ≍ hCheap (eq z)}
       → {eqB : ∀ z, hCheap z ≍ hLog (eq.symm z)}
       → ∀ z, elim hLog hCheap eqA eqB (mkA z) = (hLog z) := _
theorem elimCheap : {motive : carry → Type w}
       → {hLog : (z : A) → motive (mkA z)}
       → {hCheap : (z : B) → motive (mkB z)}
       → {eqA : ∀ z, hLog z ≍ hCheap (eq z)}
       → {eqB : ∀ z, hCheap z ≍ hLog (eq.symm z)}
       → ∀ z : B, elim hLog hCheap eqA eqB (mkB z) = (hCheap z) := _
```

Through quite a bit of work (which I call transliteration, as seen in the ABI file), You can free a universe level and open for a more general usage of the function.

3.6.1 Weak univalence

$$\begin{tikzcd}
\beta \arrow[rr, equal, "\mathrm{eq}"] \arrow[dr, "\mathrm{mkB}"'] & & \alpha \arrow[dl, "\mathrm{mkA}"] \\
& \mathrm{ABI}\ \alpha\beta &
\end{tikzcd}$$

Figure 12: Weak univalence up to shrink

3.6.2 Relation to Shrink and further universe transforming types

TODO

3.7 NTMonad

3.8 Interaction trees

TODO

3.8.1 Weak bisimilarity

TODO

3.8.2 A formally verified compiler

TODO

Chapter 4: Evaluation

Here we will

4.1 Performance between SME and PA

After building the equivelence, and instantiating shrink, we now have the ability to compare the performance between 3 representations: The SME encoding (high performance, hpRuns), the PA encoding (slow, slRuns), and a hand made big implementation (bigRuns). We compare these 3 in Figure 13. In Figure 13 we see the 3 representations specalised to infinite streams, the code for which can be seen in Listing 6

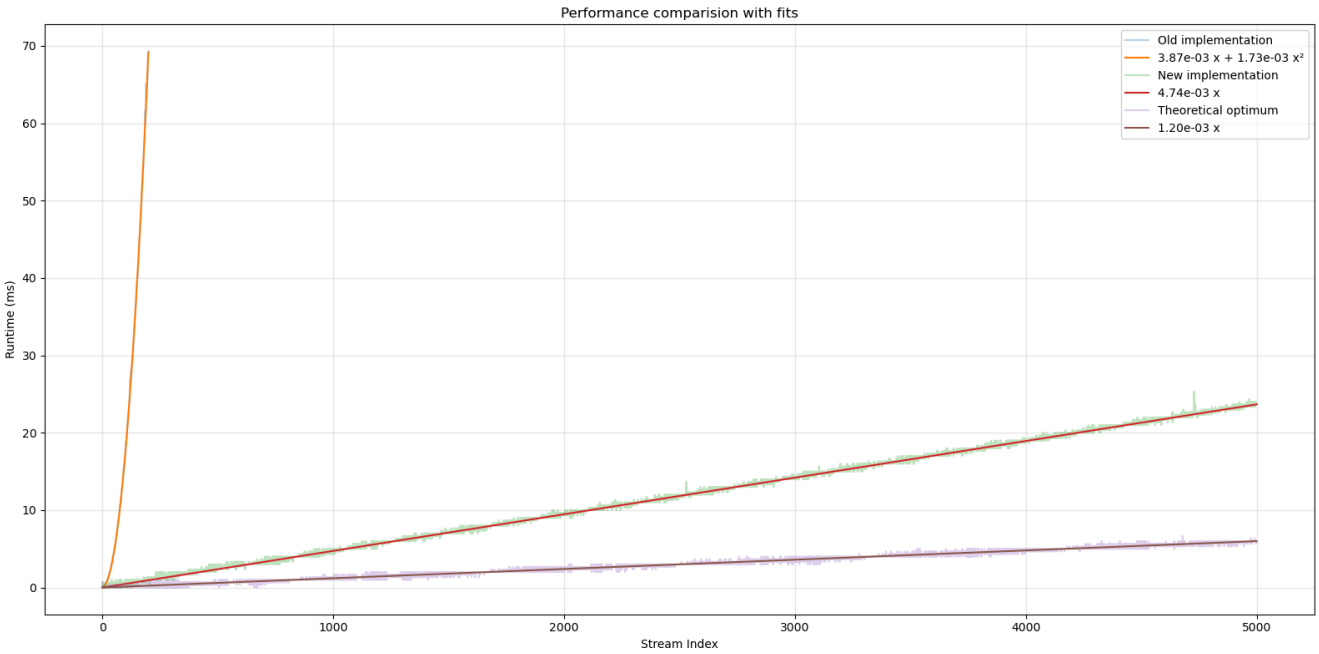


Figure 13: Performance of SME, PA, and Big representation

```
def QStream.Base : MvPFunctor 2 := ⟨
  Unit,
  fun _ _ ⇒ Unit
⟩

def QStreamSl α := M QStream.Base (fun _ ⇒ α)
def QStreamHp α := HpLuM QStream.Base (fun _ ⇒ α)

structure QStreamBig.{u} (α : Type _) where
  corec ::
    {t : Type u}
    (functor : t → Nat × t)
    (obj : t)

def numsSl : QStreamSl Nat :=
  .corec _ (fun n ⇒ ⟨.unit, fun | .fz, .unit ⇒ n.succ | .fs .fz, .unit ⇒ n⟩) Nat.zero

def numsHp : QStreamHp Nat :=
  .corec' (fun n ⇒ ⟨.unit, fun | .fz, .unit ⇒ n.succ | .fs .fz, .unit ⇒ n⟩) Nat.zero

def numsBig : QStreamBig Nat :=
  QStreamBig.corec (fun n ⇒ ⟨n, n + 1⟩) 0

def QStreamBig.getNth (x : QStreamBig Nat) : Nat → Nat
  | 0 ⇒ x.dest.fst
  | n+1 ⇒ x.dest.snd.getNth n

def QStreamSl.getNth (x : QStreamSl Nat) : Nat → Nat
  | 0 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ v (.fs .fz) .unit
  | n+1 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ QStreamSl.getNth (v .fz .unit) n

def QStreamHp.getNth (x : QStreamHp Nat) : Nat → Nat
  | 0 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ v (.fs .fz) .unit
  | n+1 ⇒ match x.dest with
    | ⟨.unit, v⟩ ⇒ QStreamHp.getNth (v .fz .unit) n
```

Chapter 5: Conclusions

Bibliography

[1] L. M. de Moura, S. Kong, J. Avigad, F. van Doorn, and J. von Raumer, "The Lean Theorem Prover (System Description)," in *Automated Deduction - CADE-25 - 25th International Conference on Automated Deduction, Berlin, Germany, August 1-7, 2015, Proceedings*, 2015, pp. 378–388. doi: 10.1007/978-3-319-21401-6_26.

[2] Marcelo P. Fiore and Jon M. Sterling, "Discrete Mathematics 23-24." https://www.cl.cam.ac.uk/teaching/2324/DiscMath/, Apr. 2026.

[3] T. Univalent Foundations Program, *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study: https://homotopytypetheory.org/book, 2013.

[4] Steve Awodey, "Cambridge Lectures on Categorical Logic and Type Theory." https://awodey.github.io/cltt/, Mar. 2026.

[5] The mathlib Community, "The Lean Mathematical Library," in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, in CPP 2020. New Orleans, LA, USA: ACM, Jan. 2020. doi: 10.1145/3372885.3373824.

[6] nLab authors, "polynomial functor." Jan. 2026.

[7] A. Keizer, "Implementing a definitional (co)datatype package in Lean 4, based on quotients of polynomial functors," Master's thesis, https://eprints.illc.uva.nl/id/eprint/2239/1/MoL-2023-03.text.pdf, 2023.

[8] J. Avigad, M. Carneiro, and S. Hudon, "Data Types as Quotients of Polynomial Functors," in *10th International Conference on Interactive Theorem Proving (ITP 2019)*, J. Harrison, J. O'Leary, and A. Tolmach, Eds., in Leibniz International Proceedings in Informatics (LIPIcs), vol. 141. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, pp. 6:1–6:19. doi: 10.4230/LIPIcs.ITP.2019.6.

[9] J. Lambek, "A Fixpoint Theorem for complete Categories.," *Mathematische Zeitschrift*, vol. 103, pp. 151–161, 1968, [Online]. Available: http://eudml.org/doc/170906

[10] J. Adámek, "Free algebras and automata realizations in the language of categories," *Commentationes Mathematicae Universitatis Carolinae*, vol. 15, no. 4, pp. 589–602, 1974, [Online]. Available: http://eudml.org/doc/16649

[11] M. P. Fiore, "A Coinduction Principle for Recursive Data Types Based on Bisimulation," *Information and Computation*, vol. 127, no. 2, pp. 186–198, 1996, doi: https://doi.org/10.1006/inco.1996.0058.

[12] B. Goodspeed, "Proof Driven Development," *CoRR*, 2015, [Online]. Available: http://arxiv.org/abs/1512.02102

[13] L.-y. Xia *et al.*, "Interaction trees: representing recursive and impure programs in Coq," *Proc. ACM Program. Lang.*, vol. 4, no. POPL, Dec. 2019, doi: 10.1145/3371119.

[14] J. Kim, Y. Nam, and C.-K. Hur, "Coco: Corecursion with Compositional Heterogeneous Productivity," *Proc. ACM Program. Lang.*, vol. 10, no. POPL, Jan. 2026, doi: 10.1145/3776733.

6 Appendicies

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur, inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut ratione et via procedat oratio. Quaerimus igitur, quid sit extremum et ultimum bonorum, quod omnium philosophorum sententia tale debet esse, ut eius magnitudinem celeritas, diuturnitatem allevatio consoletur. Ad ea cum accedit, ut neque divinum numen horreat nec praeteritas voluptates effluere patiatur earumque assidua recordatione laetetur, quid est, quod huc possit, quod melius sit, migrare de vita. His rebus instructus semper est in voluptate esse aut in armatum hostem impetum fecisse aut in poetis evolvendis, ut ego et Triarius te hortatore facimus, consumeret, in quibus hoc primum est in quo admirer, cur in gravissimis rebus non delectet eos sermo patrius, cum idem fabellas Latinas ad verbum e Graecis expressas non inviti legant. Quis enim tam inimicus paene nomini Romano est, qui Ennii Medeam aut Antiopam Pacuvii spernat aut reiciat, quod se isdem Euripidis fabulis delectari dicat, Latinas litteras oderit? Synephebos ego, inquit, potius Caecilii aut Andriam Terentii quam utramque Menandri legam? A quibus tantum dissentio, ut, cum Sophocles vel optime scripserit Electram, tamen male conversam Atilii mihi legendam putem, de quo Lucilius: 'ferreum scriptorem', verum, opinor, scriptorem tamen, ut legendus sit. Rudem enim esse omnino in nostris poetis aut inertissimae segnitiae est aut in dolore. Omnis autem privatione doloris putat Epicurus terminari summam voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in voluptate aut a voluptate discedere.

Nam cum ignoratione rerum bonarum et malarum maxime hominum vita vexetur, ob eumque errorem et voluptatibus maximis saepe priventur et durissimis animi doloribus torqueantur, sapientia est adhibenda, quae et terroribus cupiditatibusque detractis et omnium falsarum opinionum temeritate derepta certissimam se nobis ducem praebeat ad voluptatem. Sapientia enim est una, quae maestitiam pellat ex animis, quae nos exhorrescere metu non sinat. Qua praeceptrice in tranquillitate vivi potest omnium cupiditatum ardore restincto. Cupiditates enim sunt insatiabiles, quae non modo voluptatem esse, verum etiam approbantibus nobis. Sic enim ab Epicuro reprehensa et correcta permulta. Nunc dicam de voluptate, nihil scilicet novi, ea tamen, quae te ipsum probaturum esse confidam. Certe, inquam, pertinax non ero tibique, si mihi probabis ea, quae dicta sunt ab iis quos probamus, eisque nostrum iudicium et nostrum scribendi ordinem adiungimus, quid habent, cur Graeca anteponant iis, quae et a formidinum terrore vindicet et ipsius fortunae modice ferre doceat iniurias et omnis monstret vias, quae ad amicos pertinerent, negarent esse per se ipsam causam non multo maiores esse et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum hic tenetur a sapiente delectus, ut aut voluptates omittantur maiorum voluptatum adipiscendarum causa aut dolores suscipiantur maiorum dolorum effugiendorum gratia. Sed de clarorum hominum factis illustribus et gloriosis satis hoc loco dictum sit. Erit enim iam de omnium virtutum cursu ad voluptatem proprius disserendi locus. Nunc autem explicabo, voluptas ipsa quae qualisque sit, ut tollatur error omnis imperitorum intellegaturque ea, quae voluptaria, delicata, mollis habeatur disciplina, quam gravis, quam continens, quam severa sit. Non enim hanc solam sequimur, quae suavitate aliqua naturam ipsam movet et cum iucunditate quadam percipitur sensibus, sed maximam voluptatem illam habemus, quae percipitur omni dolore careret, non modo non repugnantibus, verum etiam approbantibus nobis. Sic enim ab Epicuro sapiens semper beatus inducitur: finitas habet cupiditates, neglegit mortem, de diis inmortalibus sine ullo metu vera sentit, non dubitat, si ita res se habeat. Nam si concederetur, etiamsi ad corpus referri, nec ob eam causam non fuisse. – Torquem detraxit hosti. – Et quidem se texit, ne interiret. – At magnum periculum adiit. – In oculis quidem exercitus. – Quid ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo et gravissimas res consilio ipsius et ratione administrari neque maiorem voluptatem ex infinito tempore aetatis percipi posse, quam ex hoc facillime perspici potest: Constituamus aliquem magnis, multis, perpetuis fruentem et animo et attento intuemur, tum fit ut aegritudo sequatur, si illa mala sint, laetitia, si bona. O praeclaram beate vivendi et apertam et simplicem et directam viam! Cum enim certe nihil homini possit melius esse quam Graecam. Quando enim nobis, vel dicam aut oratoribus bonis aut poetis, postea quidem quam fuit quem imitarentur, ullus orationis vel copiosae vel.

6.A Graphical p-functors

An alternative notation for Equation 2 can be given as: for a multivariate polynomial functor $P\alpha_0...\alpha_i$, with head types as variants $h_0...h_j$, and children c_{i,h_j}, it would conventionally be written as Equation 8, I will display the hs and cs as in Figure 14. I write an object of this functor, picked at h_0 with functions f_i, as seen in Figure 15. The justification of this notation will be seen Section 2.3.

$$(j' : \text{Fin } j) \times \left(h : h_{j'}\right) \times (i' : \text{Fin } i) \to c_{i',h} \to \alpha_{i'} \tag{8}$$

$$\begin{array}{c|c|c|c}
\alpha_i & c_{i,v} & ... & c_{i,v} \\
\vdots & \vdots & \iddots & \vdots \\
\alpha_0 & c_{0,v} & ... & c_{0,v} \\
\hline
 & k_0\ (v : h_0) & ... & k_j\ \left(v : h_j\right)
\end{array}$$

Figure 14: The head and child components of $P\alpha_0...\alpha_i$

$$\begin{array}{ccc}
\alpha_i & \xleftarrow{f_i} & c_{i,v} \\
\vdots & & \vdots \\
\alpha_0 & \xleftarrow{f_0} & c_{0,v}
\end{array}$$

$$\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots$$

$$k_0\ v$$

Figure 15: An object of $P\alpha_0...\alpha_i$

Mapping on MvPfunctors is done by composition of the arrows. This can be seen in Figure 16. By set forming a category, mapping a composition is the same as mapping each one by one.

$$\begin{array}{ccc}
\beta_i & \xleftarrow{g_i} & \alpha_i \\
\vdots & & \vdots \\
\beta_0 & \xleftarrow{g_0} & \alpha_0
\end{array}
\quad
\begin{array}{ccc}
\alpha_i & \xleftarrow{f_i} & c_{i,v} \\
\vdots & & \vdots \\
\alpha_0 & \xleftarrow{f_0} & c_{0,v}
\end{array}
\qquad\qquad
\begin{array}{ccc}
\beta_i & \xleftarrow{g_i \circ f_i} & c_{i,v} \\
\vdots & & \vdots \\
\beta_0 & \xleftarrow{g_0 \circ f_0} & c_{0,v}
\end{array}$$

$$\cdots\cdots\cdots\cdots\cdots\cdots\cdots\cdots

6.A.A Common pfunctors

6.A.A.A Constant

A functor, fixed at a constant type T and a arity i, denoted $\text{const}_i T$, shown in Figure 17. It has one constructor $\text{mk}_\alpha : T \to \text{const}_i T \alpha_0 ... \alpha_i$, with an inverse $\text{get}_\alpha : \text{const}_i T \alpha_0 ... \alpha_i \to T$.

α_i	0
$\vdots$	$\vdots$
α_0	0
	$\text{mk}\ (v : T)$

Figure 17: Definition of $\text{const}_i T$

$$\begin{array}{ccc} \beta_i \xleftarrow{g_i} \alpha_i & \alpha_i \xleftarrow{¡} 0 & \beta_i \xleftarrow{¡} 0 \\ \vdots \qquad \vdots & \vdots \qquad \vdots & \vdots \qquad \vdots \\ \beta_0 \xleftarrow{g_0} \alpha_0 & \alpha_0 \xleftarrow{¡} 0 & \beta_0 \xleftarrow{¡} 0 \\ \hline \text{<\$>} & \text{mk } v & = \quad \text{mk } v \\ \hline & \text{mk}_\alpha v & \text{mk}_\beta v \end{array}$$

Figure 18: By initial hom uniqueness,

the object is invariant under mapping.

6.A.A.B Prj

Given a typevector $\alpha_0 ... \alpha_i$, and a select index into the vector $(n : \text{Fin } i)$, the projection functor holds a value of type α_n. It is defined in Figure 19. It has one constructor $\text{mk}_\alpha : \alpha_n \to \text{prj}_n \alpha_0 ... \alpha_i$, with an inverse $\text{get}_\alpha : \text{prj}_n \alpha_0 ... \alpha_i \to \alpha_n$.

$$
\begin{array}{c:c:}
\alpha_i & 0 \\
\vdots & \vdots \\
\alpha_n & 1 \\
\vdots & \vdots \\
\alpha_0 & 0 \\
\cdots\cdots & \cdots\cdots \\
 & \mathrm{mk} \\
 &
\end{array}
$$

Figure 19: Definition of prj_n

$$
\begin{array}{ccccccc}
\beta_i & \xleftarrow{g_i} & \alpha_i & \alpha_i & \xleftarrow{¡} & 0 & \beta_i \xleftarrow{¡} 0 \\
\vdots & & \vdots & \vdots & & \vdots & \vdots \\
\beta_n & \xleftarrow{g_n} & \alpha_n & \alpha_n & \xleftarrow{\boldsymbol{K}v} & 1 & \beta_n \xleftarrow{\boldsymbol{K}(g_n v)} 1 \\
\vdots & & \vdots & \vdots & & \vdots & \vdots \\
\beta_0 & \xleftarrow{g_0} & \alpha_0 & \alpha_0 & \xleftarrow{¡} & 0 & \beta_0 \xleftarrow{¡} 0 \\
\cdots & & & \cdots & & & \cdots \\
 & \text{<\$>} & & & \mathrm{mk} & & = \quad \mathrm{mk} \\
 & & & \cdots & & & \cdots \\
 & & & & \mathrm{mk}_n v & & \mathrm{mk}_n(g_n v)
\end{array}
$$

Figure 20: By initial hom uniqueness, mapping only changes the nth argument.

6.A.A.C Sum

6.A.A.D Option

6.A.A.E Functions from fixed types

6.A.A.F W

6.A.A.G M

7 Proposal

See proposal overleaf

Efficient coinductives through state-machine corecursors

Supervisor: Alex Keizer

Marking supervisor: Jamie Vicary

Director of studies: Russell Moore

Word count: 993

1 Work to be undertaken

Previous work has shown how coinductive types can be encoded in Lean as quotients of polynomial functors (QPFs)[1]. This is done through progressive approximation (Section 4.1.2). For example accessing the n index of a stream takes $\mathcal{O}(n^2)$ time. This becomes an issue when using Lean as a general purpose programming language. An alternative approach is using a state-machine based encoding (Section 4.1.1) of cofixed points. With these we get nice computational behaviour of the compiled code. In Figure 1 we can see in blue the performance from the current implementation, versus in orange the performance of a prototype implementation the state-machine based version.

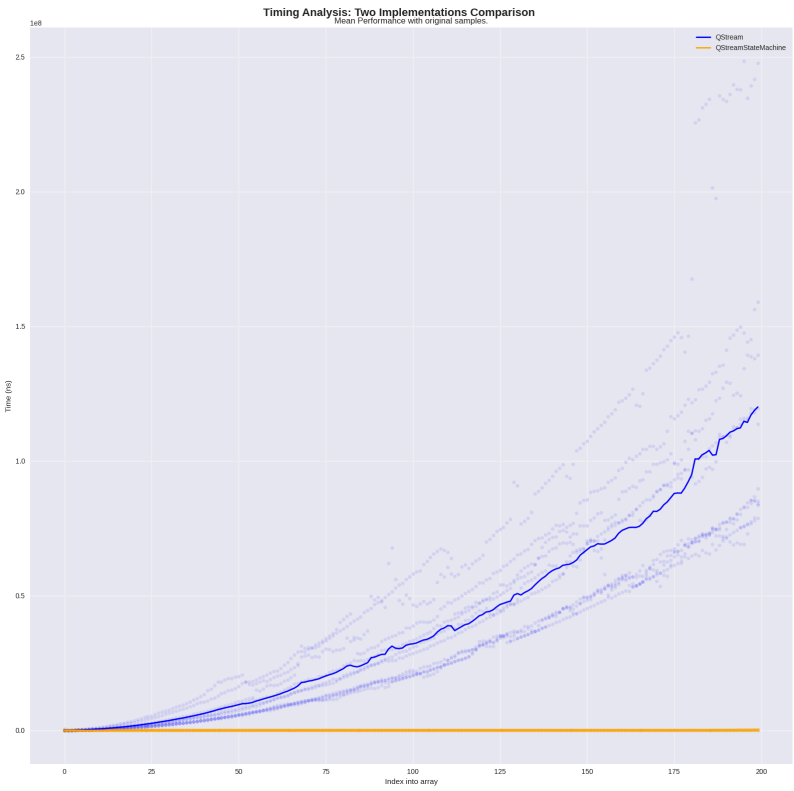


Figure 1: Graph plotting current performance v possible gains

2 Core project

This project will be implementing the state-machine encoding (Section 4.1.1) of coinductive types, and formalising the equivalence between these two embedding (Section 4.3). I will start with the special example of equivalence between `Streams` in the two representations, then building up to the general cofix structure.

After this I will be implementing a coinductive monad allowing us to encode non-termination as an effect using the state-machine representation. This will be an instance of the two representations.

3 Starting point

I have worked with QPFs on the meta-programming side for an internship between my Part Ia and Part Ib. During this I learnt of the basics of polynomial functors, and `TypeVec`s. This means I am aware of what the underlying structures are when it comes to the raw implementation. I have also done a feasibility assessment of the project by seeing how the current polynomials respond to universe levels. This lead to me making 2 pull requests (#28095, #28279) to mathlib on `TypeVec` in preparation for my project. I also tried to merge (#28112) an implementation of commutable Shrink to mathlib that later got reverted when it was found to be inconsistent.

There are more minor refactoring pull requests towards the mathlib repository which don't change any behaviour but in general all of these can be *found on this link*. I Include these for completeness and transparency.

4 Substance

Section 4.1.1, Section 4.3 and Section 4.4 are all discuss structures that will be implemented throughout the project. Section 4.1.2 and Section 4.2 both are implemented in Avigard et al [1].

4.1 The M type

The M type is the name given to the terminal coalgebra of polynomial functors; the possibly infinitely deep trees. This means the implementations must have both: A a corecursor $\text{corec} : \{\alpha : \text{Type}\} \to (f : \alpha \to F\alpha) \to \alpha \to MF$, and a destructurer $\text{dest} : \text{M}F \to F(MF)$. We have two encodings of the M type.

4.1.1 State-machine encoding

The state-machine encoding is the naïve way to implement the terminal coalgebra. Given some polynomial functor F, the state-machine encoding is given by: some type $\alpha : \text{Type}$, a function $f : \alpha \to F\alpha$, and some witness $a : \alpha$. Then you quotient over bisimilarity, thereby only allowing direct usage of $\text{dest} : \text{Sme } F \to F(\text{Sme } F)$. The baseline implementation of this should be straight-forward.

4.1.2 Progressive approximation encoding

The other way is to generate them by progressive approximation where earlier trees must "agree" with the later ones. Agreement is given by them being the same up to the previous depth as seen in Figure 2.

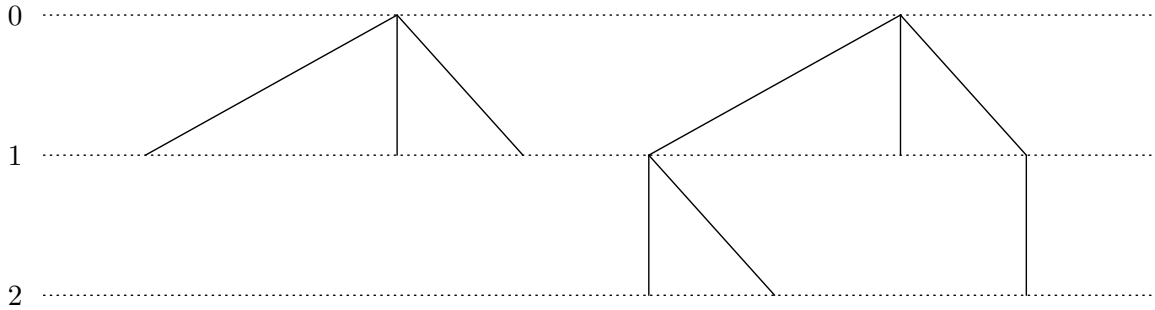


Figure 2: Agreement of two trees of height 1 and 2

4.2 The `Cofix` type

`Cofix` is the terminal coalgebra in QPFs, These are simply a quotient over M types lifting the quotient from the source QPF.

4.3 The equivalence

The equivalence between the current implementation of `Cofix` and M and the state-machine representation is the core of the project. The functions in both directions are given by $f : \mathrm{corec}_{\mathrm{Pae}}\mathrm{dest}_{\mathrm{Sme}}$ and $f^{-1} : \mathrm{corec}_{\mathrm{Sme}}\mathrm{dest}_{\mathrm{Pae}}$. The difficulty comes from proving $f \circ f^{-1} = \mathbb{1}$ and $f^{-1} \circ f = \mathbb{1}$. The proof of these equalities will be proven using bisimilarity, and other parts of the theory currently established for M types. Simpler versions of this equivalence also exist for the simpler representation.

4.4 The non-termination monad

The non-termination monad is a simple example of a coinductive. This will be useful for testing the state-machine representation's performance. The structure of the monad as a coinductive has two constructors as seen in this psudocode:

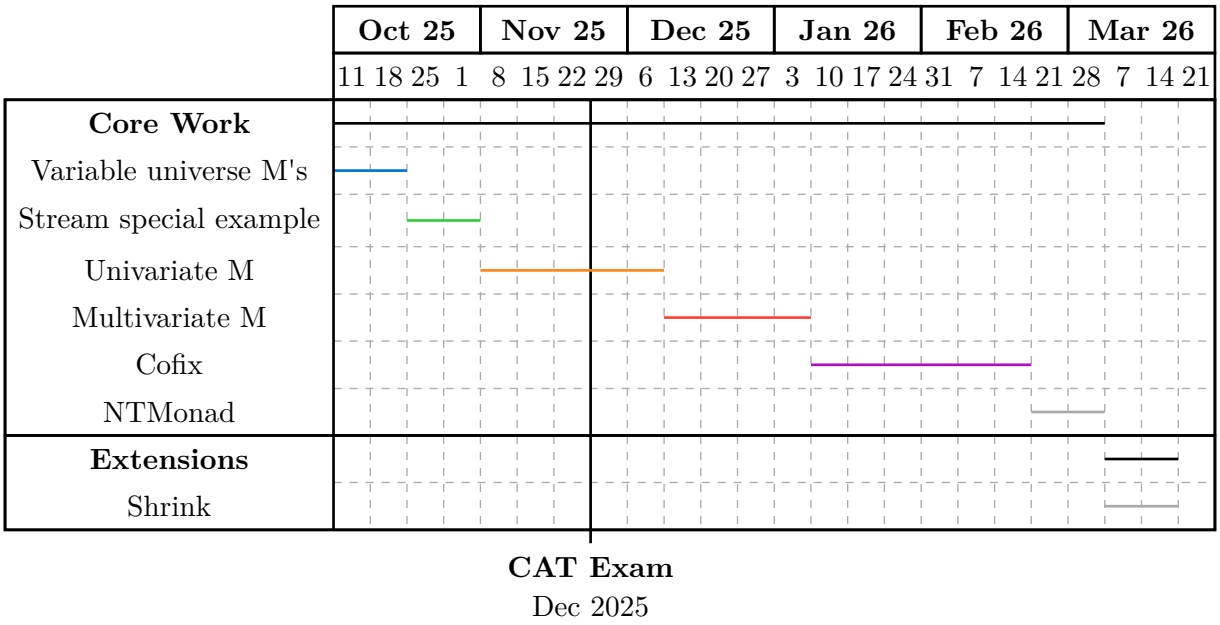
```
coinductive NTMonad (A : Type)
  | val : A -> NTMonad A
  | tau : NTMonad A -> NTMonad A
```

5 Evaluation

This projects success can be broken down into a few disjoint criteria.

1. If the equivalence (Section 4.3) has been proven,
2. how close to the theorised performance we get with the state-machine encoding,
3. and the ability to construct the example structures.

6 Core timeline



The plan for work would be divided into a few different stages.

6.1 Variable universe `Ms` (2025-10-11 2w 2025-10-24)

To begin with I have to make the corecursor universe polymorphic. As stated in Section 3 I have most of this code written. I will attempt to get this merged into mathlib, but if this review process takes too long I will be working on my own branch.

6.2 `Stream` special example (2025-10-25 2w 2025-11-7)

The first step would be implementing `Stream` under the two representations. Then I will familiarise myself with using bisimilarity to formalise the equivalence. This is a first step to just understand how these features work.

6.3 Univariate `M` (2025-11-8 2w 2025-11-21 + (CAT exam) + 2025-12-05 3w 2025-12-23)

Implementing the univariate `M` type is the natural next step from this. This will be the natural next step in difficulty. The work should lay the groundwork for the next proofs.

6.4 Multivariate `M` (2025-12-26 4w 2026-01-23)

The next step will be generalising the univariate implementation, the same structure of the proof should be the same for the multivariate case. The main difficulty here comes from working with `TypeVec`s. These are quite difficult to reason about.

6.5 `Cofix` (2026-01-24 6w 2026-03-06)

Finally, the `Cofix` type has to be implemented. This will be the most challenging part as I will have to work with quotients.

6.6 Implementing the NTMonad (2026-03-07 2w 2026-03-20)

This will be a demonstration of all of the previous work.

7 Extensions

7.1 Shrinking the representations (2026-03-21 2w 2026-04-04)

This involves finding a sound way allowing the current theory to use the efficient representation. The soundness of this will be difficult to reason about as it requires working with the Lean code generator. Therefore I leave this as an extension.

8 Resources

Access to the restricted side of the lab is needed for working with the greater team.

Bibliography

[1] J. Avigad, M. Carneiro, and S. Hudon, "Data Types as Quotients of Polynomial Functors," in *10th International Conference on Interactive Theorem Proving (ITP 2019)*, J. Harrison, J. O'Leary, and A. Tolmach, Eds., in Leibniz International Proceedings in Informatics (LIPIcs), vol. 141. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, pp. 6:1–6:19. doi: 10.4230/LIPIcs.ITP.2019.6.